

SKILL WEEK - 5

QUESTION 1: Single-Quote Handling in Bash

Aim

To understand and implement **single-quote handling in Bash** for preserving literal content, preventing variable expansion, storing quoted strings, validating parsing results, and testing different edge cases.

Theory

In Bash, **single quotes (' ')** are used to preserve the literal meaning of the characters enclosed within them. When text is placed inside single quotes, Bash does not perform **variable expansion, command substitution, or interpretation of special characters**.

For example:

```
name="chakri"  
echo '$name'
```

produces:

```
$name
```

whereas:

```
echo "$name"
```

produces:

```
chakri
```

This difference is important when handling strings that contain special characters, variables, or commands that should not be interpreted by the shell.

Important points:

- Text inside single quotes is treated **literally**.
- Variables such as `$name` are **not expanded**.
- Command substitution such as `$(date)` is **not executed**.
- Special characters retain their literal meaning.
- Single quotes are useful when exact text needs to be preserved.
- A single quote cannot normally be placed directly inside another single-quoted string.

1. Single Quotes vs Double Quotes in Terminal

```

chakrika@blue:~/OSSP/week5_s$ pwd
/home/chakrika/OSSP/week5_s
chakrika@blue:~/OSSP/week5_s$ name="chakri"
chakrika@blue:~/OSSP/week5_s$ echo 'Hello $name'
Hello $name
chakrika@blue:~/OSSP/week5_s$ echo "Hello $name"
Hello chakri

```

Output

Hello \$name

Hello chakri

Observation

The single-quoted string prints \$name exactly as written because **variable expansion is disabled** inside single quotes.

The double-quoted string expands \$name and prints its stored value, chakri.

Comparison

Single Quotes	Double Quotes
'...'	"..."
Preserves literal content	Allows variable expansion
\$name remains \$name	\$name becomes its value
Command substitution is not performed	Command substitution can be performed
Useful for exact/literal text	Useful when variables need to be included

2. Single Quotes Preserve Literal

Content

Content Terminal

```

chakika@@blue:~/OSP/week5_s$ echo '$HOME'
$HOME
chakrika@blue:~/OSP/week5_s$ echo '$USER'
$USER

```

Output

\$HOME

\$USER

Observation:

The contents inside single quotes are treated literally. Variables and command substitutions are not expanded.

3. Storing Quoted Strings

Variables Terminal

```
chakrika@blue:~/OSSP/week5_s$ message='OSSP Week 5 Skill Assignm
chakrika@blue:~/OSSP/week5_s$ echo "$message"
OSSP Week 5 Skill Assignment
chakrika@blue:~/OSSP/week5_s$ message='Linux shell script'
chakrika@blue:~/OSSP/week5_s$ echo "$message"
Linux shell script
chakrika@blue:~/OSSP/week5_s$ echo '$message'
$message
```

Output

OSSP Week 5 Skill Assignment

Linux shell script

\$message

4. Parsing

Validation Terminal

Aim

To understand how Bash parses quoted strings and distinguishes literal content from variables and commands.

Theory

Parsing is the process by which the shell reads and interprets a command before executing it. Quotes affect how Bash divides and interprets the command.

Single quotes tell Bash to treat everything inside them as literal text, while double quotes allow variable expansion.

```
chakrika@blue:~/OSSP/week5_s$ nano valid.sh
chakrika@blue:~/OSSP/week5_s$ bash -n valid.sh
chakrika@blue:~/OSSP/week5_s$ bash valid.sh
Hello chakri
chakrika@blue:~/OSSP/week5_s$ nano invalid.sh
chakrika@blue:~/OSSP/week5_s$ bash -n invalid.sh
invalid.sh: line 1 : unexpected EOF while looking for matching `''
chakrika@blue:~/OSSP/week5_s$ █
```

Output

For valid quoted command:

Hello amru

For an invalid quoted command:

invalid.sh: line 1: unexpected EOF while looking for matching `''

Observation:

bash -n checks the syntax without executing the script and identifies unmatched quotation marks.

Result

Single-quote handling was successfully implemented. Literal content was preserved, variable expansion was prevented, quoted strings were stored and tested, and parsing errors caused by incorrect quotation were identified.

QUESTION 2

To Apply Double Quotes, Preserve Spaces, Allow Variable Expansion, Parse Nested Tokens, Validate Outputs, Test Quoted Commands

Aim

To understand and implement **double-quote handling in Bash** for preserving spaces, allowing variable expansion, parsing nested tokens, validating outputs, and executing quoted commands.

Program / Commands

1. Double Quotes and Variable Expansion

Terminal

```
chakrika@blue :~/~/OSP/week5_s$ name="chakrika"
chakrika@blue :~/~/OSP/week5_s$ echo "Hello $name"
Hello chakrika
```

Hello chakrika

Observation:

Unlike single quotes, double quotes allow variables such as \$name to be expanded.

2. Preserve Spaces with Double Quotes

Terminal

```
chakrika@blue:~/OSP/week5_s$ text="Linux Shell Programming"
chakrika@blue:~/OSP/week5_s$ printf '<%s>\n' "$text"
<Linux Shell Programming>
chakrika@blue:~/OSP/week5_s$ printf '<%s>\n' $text
<Linux>
<Shell>
<Programming>
```

Output

<Linux Shell Programming>

<Linux>

<Shell>

<Programming>

Observation:

Using "\$text" preserves the complete string as one argument, including its spaces.

3. Argument Parsing

Create:

```
~/OSSP/week5_s$ show_args() {  
    echo "Number of arguments: $#"  
    for arg in "$@"; do  
        echo "Argument: [$arg]"  
    done  
}
```

Now:

```
chakrika@blue:~/OSSP/week5_s$ show_args "$text"  
Number of arguments: 1  
Argument: [Linux Shell Programming]  
chakrika@blue:~/OSSP/week5_s$ show_args $text  
Number of arguments: 3  
Argument: [Linux]  
Argument: [Shell]  
Argument: [Programming]
```

Output

With double quotes:

Number of arguments: 1

Argument: [Linux Shell Programming]

Without double quotes:

Without double quotes:

Number of arguments: 3

Argument: [Linux]

Argument: [Shell]

Argument: [Programming]

Observation:

Double quotes preserve the spaces and pass the complete value as a single argument.

4. Nested Tokens / Command Substitution

Terminal

```
~/OSSP/week5_s$ echo "Current user: $USER"
echo "Current directory: $(pwd)"
echo "Current date: $(date)"
```

Output

Observation:

Variables are expanded inside double quotes, and command substitution using `$(...)` is also evaluated.

5. Quoted Command

Terminal

```
chakrika@blue:~/OSSP/week5_s$ command='echo "Hello from a quoted cuoted command"'
chakrika@blue:~/OSSP/week5_s$ echo "$command"
echo "Hello from a quoted command"
chakrika@blue:~/OSSP/week5_s$ bash -c "$command"
Hello from a quoted command
```

Output

```
Current user: chakrika
Current directory: /home/chakrika/OSSP/week5_s
Current date: Sat Sep 19 03:37:33 UTC 2026
```

echo "Hello from a quoted command"

Hello from a quoted command

Observation:

The command can be stored as a quoted string and then executed using `bash -c`.

6. Validate Output / Parsing

Then execute:

```
Hello $name
Hello amru
$HOME
$USER
<Linux Shell Programming>
<Linux>
<Shell>
<Programming>
Current user: amrutha
Current directory: /home/amrutha/OSSP/week5_s
Current date: Sat Sep 19 04:01:40 UTC 2026
echo "Hello from a quoted command"
Hello from a quoted command
```

Result

Double-quote handling was successfully implemented. Spaces were preserved, variables were expanded, nested tokens and command substitution were evaluated, quoted commands were executed, and the Bash script was validated using `bash -n`.

SKILL WEEK - 6

Experiment 1: Process Escape Sequences and Special Character Handling

Aim

To develop a program that handles **process escape sequences**, **escaped spaces**, **special symbols**, and **preserved characters**, and to validate the parser output using different complex inputs.

Theory

In a command-line environment, **special characters** have specific meanings to the shell or command parser. Characters such as spaces, quotes, backslashes, pipes (|), redirection symbols (>, <), and ampersands (&) may affect how a command is interpreted.

An **escape sequence** is used to change the normal interpretation of a character. The **backslash (\)** is commonly used as an escape character. It allows special characters to be treated as ordinary characters instead of being interpreted by the shell.

For example, an escaped space:

```
Hello\ World
```

is treated as a single argument:

```
Hello World
```

Similarly, special symbols can be escaped:

```
echo\ \&\ test
```

so that the & is treated as a normal character.

A command parser must correctly identify arguments while preserving characters that have been escaped. It should distinguish between normal spaces that separate arguments and escaped spaces that belong inside an argument.

Testing the parser with simple and complex inputs helps verify that escape sequences and special characters are handled correctly.

Program:

```
C skill6_exp1.c
1  #include <stdio.h>
2  #include <string.h>
3
4  void parse_command(char *command)
5  {
6      char argument[100];
7      int i = 0, j = 0;
8      int escaped = 0;
9
10     printf("Parsed Arguments:\n");
11
12     while (command[i] != '\0')
13     {
14         if (escaped)
15         {
16             argument[j++] = command[i];
17             escaped = 0;
18         }
19         else if (command[i] == '\\')
20         {
21             escaped = 1;
22         }
23         else if (command[i] == ' ')
24         {
25             if (j > 0)
26             {
27                 argument[j] = '\0';
28                 printf("%s\n", argument);
29                 j = 0;
30             }
31         }
32         else
33         {
34             argument[j++] = command[i];
35         }
36
37         i++;
```

```

37         i++;
38     }
39
40     if (j > 0)
41     {
42         argument[j] = '\0';
43         printf("%s\n", argument);
44     }
45 }
46
47 int main()
48 {
49     char command[200];
50
51     printf("Enter command: ");
52     fgets(command, sizeof(command), stdin);
53
54     command[strcspn(command, "\n")] = '\0';
55
56     parse_command(command);
57
58     return 0;
59 }

```

Output:

```

chakrika@Chakrika:/mnt/c/Users/panak/Downloads/OSSP PRACTICALS$ cd "/mnt/c/Users/panak/Downloads/OSSP PRACTICALS"
chakrika@Chakrika:/mnt/c/Users/panak/Downloads/OSSP PRACTICALS$ ls
counter.c      counter_mutex.exe      inode_commands.txt     mmapfile.txt           producer_consumer.exe
counter.exe    deadlock_prevention.c  mmap_file               original.txt            skill6_exp1.c
counter_mutex.c  deadlock_prevention.exe mmap_file.c             producer_consumer.c     skill6_exp2.c
chakrika@Chakrika:/mnt/c/Users/panak/Downloads/OSSP PRACTICALS$ gcc skill6_exp1.c -o skill6_exp1
chakrika@Chakrika:/mnt/c/Users/panak/Downloads/OSSP PRACTICALS$ ./skill6_exp1
Enter command: echo Hello World
Parsed Arguments:
echo
Hello
World
chakrika@Chakrika:/mnt/c/Users/panak/Downloads/OSSP PRACTICALS$ ./skill6_exp1
Enter command: echo Hello\ World
Parsed Arguments:
echo
Hello World
chakrika@Chakrika:/mnt/c/Users/panak/Downloads/OSSP PRACTICALS$ ./skill6_exp1
Enter command: echo Hello\ \&\ Welcome
Parsed Arguments:
echo
Hello & Welcome
chakrika@Chakrika:/mnt/c/Users/panak/Downloads/OSSP PRACTICALS$

```

Result / Conclusion

The command parser was successfully implemented to handle **escape sequences**, **escaped spaces**, and **special characters**. The parser correctly preserved escaped characters and separated normal command arguments. The program was tested with different inputs to validate its output.

Experiment 2: Creating Child Processes

and Executing Programs

Aim

To create **child processes**, execute external programs, pass arguments, handle execution errors, and manage the parent process using system calls such as `fork()`, `exec()`, and `wait()`.

Theory

A **process** is a program that is currently being executed. In Linux, a parent process can create a new process called a **child process** using the `fork()` system call.

The `fork()` system call creates a copy of the calling process. After `fork()`:

- The **parent process** receives the child's process ID.
- The **child process** receives a return value of 0.
- A negative value indicates that process creation failed.

The `exec()` family of functions is used by a process to **replace its current program with another program**. Functions such as `execl()`, `exec1p()`, and `execvp()` can be used to execute external commands and pass arguments.

The `wait()` system call allows the parent process to wait until its child process finishes execution. This helps the parent properly manage the child process and obtain its termination status.

If an `exec()` call fails, it returns and an error can be displayed using `perror()`.

Thus, the combination of `fork()`, `exec()`, and `wait()` demonstrates how Linux creates, executes, and manages processes.

Program:

```
C skill6_exp2.c
1  #include <stdio.h>
2  #include <stdlib.h>
3  #include <unistd.h>
4  #include <sys/wait.h>
5
6  int main()
7  {
8      pid_t pid;
9      int status;
10
11     pid = fork();
12
13     if (pid < 0)
14     {
15         perror("fork failed");
16         return 1;
17     }
18
19     if (pid == 0)
20     {
21         printf("Child Process Created\n");
22         printf("Child PID: %d\n", getpid());
23
24         execlp("ls", "ls", "-l", NULL);
25
26         perror("exec failed");
27         exit(1);
28     }
29     else
30     {
31         printf("Parent Process\n");
32         printf("Parent PID: %d\n", getpid());
33
34         wait(&status);
35
36         printf("Child process completed.\n");
37     }
38
39     return 0;
40 }
```

Output:

```

chakrika@Chakrika:/mnt/c/Users/panak/Downloads/OSSP PRACTICALS$ gcc skill6_exp2.c -o skill6_exp2
chakrika@Chakrika:/mnt/c/Users/panak/Downloads/OSSP PRACTICALS$ ./skill6_exp2
Parent Process
Child Process Created
Parent PID: 804
Child PID: 805
total 489
-rwxrwxrwx 1 chakrika chakrika 607 Sep 28 06:03 counter.c
-rwxrwxrwx 1 chakrika chakrika 104290 Sep 28 06:03 counter.exe
-rwxrwxrwx 1 chakrika chakrika 795 Sep 28 06:24 counter_mutex.c
-rwxrwxrwx 1 chakrika chakrika 104308 Sep 28 06:06 counter_mutex.exe
-rwxrwxrwx 1 chakrika chakrika 1633 Sep 28 05:54 deadlock_prevention.c
-rwxrwxrwx 1 chakrika chakrika 105287 Sep 28 05:54 deadlock_prevention.exe
-rwxrwxrwx 1 chakrika chakrika 221 Sep 28 06:34 inode_commands.txt
-rwxrwxrwx 1 chakrika chakrika 16352 Sep 28 06:40 mmap_file
-rwxrwxrwx 1 chakrika chakrika 822 Sep 28 06:38 mmap_file.c
-rwxrwxrwx 1 chakrika chakrika 16 Sep 28 06:41 mmapfile.txt
-rwxrwxrwx 1 chakrika chakrika 28 Sep 28 06:33 original.txt
-rwxrwxrwx 1 chakrika chakrika 1633 Sep 28 05:53 producer_consumer.c
-rwxrwxrwx 1 chakrika chakrika 105285 Sep 28 05:53 producer_consumer.exe
-rwxrwxrwx 1 chakrika chakrika 16224 Sep 28 07:02 skill6_exp1
-rwxrwxrwx 1 chakrika chakrika 1059 Sep 28 07:00 skill6_exp1.c
-rwxrwxrwx 1 chakrika chakrika 16312 Sep 28 07:04 skill6_exp2
-rwxrwxrwx 1 chakrika chakrika 662 Sep 28 07:01 skill6_exp2.c
Child process completed.
chakrika@Chakrika:/mnt/c/Users/panak/Downloads/OSSP PRACTICALS$ █

```

Result / Conclusion

The child process was successfully created using `fork()`. The child process executed the `ls -l` command using `execlp()`, while the parent process waited for the child using `wait()`. The experiment demonstrated **process creation, program execution, argument passing, error handling, and parent-child process management.**